



Amazon DynamoDB — Complete Notes

Serverless NoSQL · Single-digit-millisecond key-value & document DB

AWS DynamoDB Type NoSQL Model Serverless Latency Single digit ms

A reader-friendly, all-in-one reference for Amazon DynamoDB.

Table of Contents

1. What is DynamoDB
2. Core Concepts
3. Primary Keys & Partitions
4. Secondary Indexes (GSI vs LSI)
5. Capacity Modes (RCU/WCU)
6. Reading & Writing Data
7. Streams & TTL
8. Global Tables & DAX
9. Transactions
10. Data Modeling (Single-Table)
11. Security
12. DynamoDB vs RDS
13. Pricing
14. Common CLI Commands
15. Best Practices
16. Quick Mental Model
17. Common Interview Questions

1. ⚡ What is DynamoDB

Amazon DynamoDB is a **fully managed, serverless NoSQL database** delivering **single-digit-millisecond** performance at any scale. There are no servers to manage, it scales automatically, and it's built for massive throughput with consistent low latency. It stores **key-value** and **document** data.

💡 **Mental model:** DynamoDB is a giant, infinitely scalable hash table. You design around **access patterns and keys** — not normalized tables and joins. Performance stays flat whether you have 1 MB or 100 TB.

Common uses: high-scale web/mobile backends, gaming leaderboards, shopping carts, session stores, IoT, event sourcing, anything needing predictable low latency at scale.

2. 🧩 Core Concepts

Concept	Description
Table	A collection of items (no fixed schema beyond the key).
Item	A single record (like a row); up to 400 KB .
Attribute	A field of an item (like a column); items can differ.
Primary Key	Uniquely identifies an item — partition key, optionally + sort key.
Partition Key (PK)	Hash key determining which partition stores the item.
Sort Key (SK)	Optional; orders items within a partition.
Secondary Index	Alternate query path (GSI / LSI).
RCU / WCU	Read / Write Capacity Units (throughput).
Stream	Ordered change log of item modifications.

3. 🔑 Primary Keys & Partitions

Two primary-key types:

Type	Made of	Behavior
Simple (Partition key)	Partition key only	Key must be unique; item located by hashing the PK.
Composite (Partition + Sort key)	PK + SK	Many items can share a PK; SK orders them and enables range queries.

- DynamoDB hashes the **partition key** to pick a **physical partition**. Spread keys evenly to avoid a **hot partition** (one key getting disproportionate traffic).
- A composite key enables powerful **Query** patterns: "get all items with PK= `user#123` and SK between two values."

⚠️ **Hot partition / hot key** is the classic DynamoDB pitfall — choose a high-cardinality partition key so load spreads across partitions.

4. 📦 Secondary Indexes (GSI vs LSI)

Indexes let you query on non-primary-key attributes.

	GSI (Global Secondary Index)	LSI (Local Secondary Index)
Partition key	Different from the table	Same as the table; different sort key
When created	Anytime	Only at table creation
Capacity	Its own RCU/WCU	Shares the table's capacity
Consistency	Eventual only	Supports strong reads
Limit	20 per table (default)	5 per table

Use a **GSI** for most "query by another attribute" needs (flexible, added anytime). Use an **LSI** only when you need an alternate sort key with strong consistency, decided up front.

5. 🇮🇹 Capacity Modes (RCU/WCU)

Mode	How	Best for
On-Demand	Pay per request, instant scaling, no planning	Spiky/unknown traffic, new apps
Provisioned	You set RCU/WCU (with optional auto scaling)	Predictable traffic, lowest cost at steady load

- **RCU** = 1 strongly-consistent read/sec of up to 4 KB (or 2 eventually-consistent reads).
- **WCU** = 1 write/sec of up to 1 KB.
- Exceeding provisioned capacity → **throttling** (`ProvisionedThroughputExceeded`); the SDK retries with backoff. Auto scaling adjusts capacity to a target utilization.

6. 🔍 Reading & Writing Data

Operation	Does
PutItem / UpdateItem / DeleteItem	Write/modify/remove an item.
GetItem	Fetch one item by full primary key.
Query	Fetch items by partition key (+ optional sort-key condition). Efficient.
Scan	Read the entire table with filters. Expensive — avoid at scale.
BatchGet / BatchWrite	Up to 25 items / 100 keys per call.

- **Consistency**: reads are **eventually consistent** by default (cheaper, ~1s lag possible); request **strongly consistent** reads when you need the latest write.
- **Query** uses the key; **filters** are applied *after* the read (you still pay for scanned items). Prefer Query over Scan.

7. 🔄 Streams & TTL

- **DynamoDB Streams** — an ordered, 24-hour change log of item-level writes. Trigger a **Lambda** to react (replication, aggregation, notifications, event sourcing).
 - **TTL (Time To Live)** — set an expiry timestamp attribute; DynamoDB **auto-deletes expired items** for free (background process, within ~48h). Great for sessions, temp data.
-

8. 🌐 Global Tables & DAX

- **Global Tables** — multi-region, **active-active** replication for low-latency global access and disaster recovery (last-writer-wins conflict resolution).
 - **DAX (DynamoDB Accelerator)** — a fully managed **in-memory cache** in front of DynamoDB, taking read latency from milliseconds to **microseconds** for read-heavy workloads (no app rewrite for the DynamoDB API).
-

9. 🔒 Transactions

- **TransactWriteItems / TransactGetItems** give **ACID** transactions across up to 100 items / multiple tables — all-or-nothing.
 - **Conditional writes** (`ConditionExpression`) enable optimistic locking and idempotency (e.g. "only write if this attribute doesn't exist").
 - **Atomic counters** (`ADD`) increment values without read-modify-write races.
-

10. 🏗️ Data Modeling (Single-Table)

- DynamoDB rewards **modeling around access patterns first**, not entities. The advanced pattern is **single-table design**: many entity types in one table, distinguished by composite keys + a GSI.
- **Denormalize** and duplicate data to satisfy reads in one Query (no joins).
- Use **key prefixes** like `USER#123 / ORDER#456` and **overloaded GSIs** to serve multiple query patterns.

💡 The mantra: *"Know your queries before you design your table."* Joins and ad-hoc queries are RDBMS strengths — in DynamoDB you precompute them into the key design.

11. 🛡️ Security

- **IAM** policies control access — can be scoped down to **item/attribute level** with conditions (fine-grained access control).
 - **Encryption at rest** is **on by default** (KMS); **TLS** in transit.
 - Access privately from a VPC via a **Gateway VPC endpoint** (free, like S3).
 - **PITR (Point-In-Time Recovery)** restores to any second in the last 35 days; on-demand backups persist independently.
-

12. ⚖️ DynamoDB vs RDS

	DynamoDB	RDS / Aurora
Model	NoSQL key-value/document	Relational (SQL)
Schema	Flexible (per item)	Fixed schema
Scale	Virtually unlimited, automatic	Vertical + read replicas
Queries	By key / index (no joins)	Rich SQL, joins, ad-hoc
Latency	Single-digit ms at any scale	Good, but tuning needed at scale
Best for	High-scale, known access patterns	Complex queries, transactions, reporting

13. 💰 Pricing

You pay for:

1. **Throughput** — On-Demand (per million read/write request units) or Provisioned (RCU/WCU per hour).
2. **Storage** — per GB-month of data + indexes.
3. **Extras** — Streams reads, Global Tables replicated writes, DAX nodes, PITR/backups, data transfer out.

🔗 **Cost tips:** avoid **Scans** (they consume capacity for every item) · use On-Demand for spiky traffic, Provisioned + auto scaling for steady · project only needed attributes into GSIs · set **TTL** to purge stale data · pick a good partition key to dodge throttling.

14. 📖 Common CLI Commands

```
aws dynamodb create-table --table-name Users \
  --attribute-definitions AttributeName=userId,AttributeType=S \
  --key-schema AttributeName=userId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST

aws dynamodb put-item --table-name Users \
  --item '{"userId":{"S":"123"},"name":{"S":"Alice"}}'

aws dynamodb get-item --table-name Users --key '{"userId":{"S":"123"}}'

aws dynamodb query --table-name Users \
  --key-condition-expression "userId = :id" \
  --expression-attribute-values '{"id":{"S":"123"}}'

aws dynamodb update-item --table-name Users --key '{"userId":{"S":"123"}}' \
  --update-expression "SET #n = :v" \
  --expression-attribute-names '{"#n":"name"}' \
  --expression-attribute-values '{"v":{"S":"Bob"}}'

aws dynamodb scan --table-name Users          # avoid at scale
```

15. Best Practices

- **Design around access patterns**; prefer **Query** over **Scan**.
 - Choose a **high-cardinality partition key** to spread load (avoid hot partitions).
 - Use **GSIs** for alternate queries; project only the attributes you need.
 - Use **On-Demand** for unpredictable load, **Provisioned + auto scaling** for steady.
 - Add **TTL** for ephemeral data; enable **PITR** for safety.
 - Use **conditional writes / transactions** for correctness; **DAX** for read-heavy hotspots.
 - Keep items small (< 400 KB); store large blobs in **S3** and reference them.
-

16. Quick Mental Model

- **DynamoDB = serverless NoSQL, single-digit-ms, infinite scale.** No servers, no joins.
 - An item lives in a **partition** chosen by hashing the **partition key**; a **sort key** orders items and enables range queries.
 - Query by key/index — **avoid Scans**. Reads are eventually consistent unless you ask for strong.
 - **GSI** (any key, eventual, own capacity) vs **LSI** (same PK, strong, created up front).
 - **On-Demand** vs **Provisioned (RCU/WCU)** capacity; throttling if you exceed provisioned.
 - **Streams** trigger Lambda on changes; **TTL** auto-expires items; **Global Tables** go multi-region; **DAX** caches reads.
 - **Model around your queries** — denormalize, design keys first.
-

17. Common Interview Questions

Rapid-fire questions interviewers ask about DynamoDB:

- **Q: When DynamoDB over RDS?** — When you need massive scale, predictable low latency, flexible schema, and your access patterns are known/key-based — not complex joins or ad-hoc SQL.
 - **Q: Query vs Scan?** — Query reads by partition key (efficient, uses the index). Scan reads the whole table and filters after — expensive; avoid at scale.
 - **Q: GSI vs LSI?** — GSI = different partition key, added anytime, own capacity, eventual reads. LSI = same partition key + alternate sort key, created only at table creation, strong reads.
 - **Q: What's a hot partition?** — When one partition key gets disproportionate traffic, throttling that partition. Fix with a higher-cardinality / well-distributed key.
 - **Q: Strong vs eventual consistency?** — Default reads are eventually consistent (cheaper, slight lag). Strongly consistent reads return the latest write at 2x the RCU cost.
 - **Q: On-Demand vs Provisioned capacity?** — On-Demand auto-scales per request (spiky/unknown load). Provisioned sets RCU/WCU (cheaper at steady load) with optional auto scaling.
 - **Q: How do you react to data changes?** — DynamoDB **Streams** → trigger a Lambda (replication, aggregation, events).
 - **Q: How are transactions handled?** — TransactWriteItems/TransactGetItems give ACID across up to 100 items; conditional writes give optimistic locking/idempotency.
 - **Q: How do you cache DynamoDB reads?** — **DAX**, an in-memory cache giving microsecond reads for read-heavy workloads.
-

 *Notes compiled as a quick reference for Amazon DynamoDB.*